ARTEMIS V2

Boosting Milo Performance

Kesanapalli Sai Rishitha
Product Intern
(May 12 ,2025 – July 18 , 2025)

⇒ Table of Contents

GitHub repo: https://github.com/Rishiithaaa/milo

Adobe's Web Development Framework:

- Component-based block architecture powering Adobe's Franklin-based websites
- Modular, reusable design system with built-in internationalization and analytics
- Follows a content-first approach for fast and scalable web development
- Primarily uses client-side rendering, affecting performance and SEO
- Integrates personalization, A/B testing, and Adobe services for dynamic user experiences

References: https://deepwiki.com/adobecom/milo

PROBLEM STATEMENT

Client-Side Rendering

- The browser downloads a **minimal HTML**, loads **JavaScript**, and then builds and renders the **entire content** on the **client (user's browser)**.

References: https://developer.mozilla.org/en-US/docs/Glossary/CSR

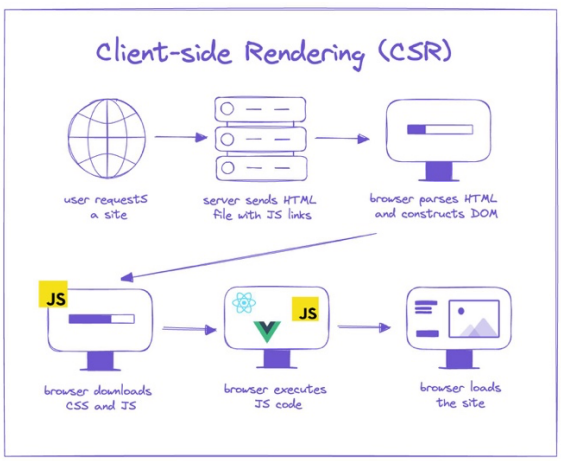
Functionality

User enters URL → Server returns index.html with JS links

Browser displays blank screen → Downloads and executes JS

[App mounts to DOM → Fetches data from APIs]

[JS renders components dynamically → User sees interactive page]



Why Client-Side Rendering Fails for Performance-Critical, SEO-Dependent Web Apps

---

1. Slower Initial Load

What happens in Milo's Client-Side Rendering:

1. **Browser receives minimal but meaningful HTML** (e.g., photoshop.plain.html) — block elements like , <section>, etc., are present, but lack behavior or enhanced layout.
2. **Triggers a cascade of network requests** for block-specific JS, CSS, and data. Although Milo avoids large monolithic bundles, the block system loads scripts sequentially per section, increasing cumulative load time.
3. **JavaScript is parsed and executed block-by-block**, enhancing each section in order .
4. **Even with static HTML present**, many blocks rely on dynamic data via APIs — delaying full interactivity.
5. **Only after all of this** is the page fully interactive with animations, hover effects, and layout finalization.

Why this is problematic:

- The user sees a **blank screen or spinner** longer than expected.
- **Interactivity is delayed**, especially on first section or global nav.
- Performance metrics such as:
  - **FCP** (First Contentful Paint)
  - **LCP** (Largest Contentful Paint)
  - **TTI** (Time to Interactive) are negatively impacted.

Real-world impact:

On slow connections or low-end devices, **LCP/FCP can exceed 4s**, leading to a "Poor" rating in **Core Web Vitals**, ultimately hurting SEO and user experience.

---

2. SEO Limitations

What happens in Milo's Client-Side Rendering:

- The server responds with minimal but meaningful HTML (e.g., photoshop.plain.html) containing structural elements and block wrappers.
- Main content and layout enhancements load only after JavaScript execution.

Why this is problematic for SEO:

- Many search engine crawlers (especially those other than Google) do not reliably execute JavaScript or wait for full page enhancement.
- As a result:
  - Critical text content may never appear to the bot.
  - Pages risk being indexed without headings, descriptions, or product information

- **Impact:**
  Pages depending on Milo's default CSR can suffer from **incomplete indexing**, affecting **pages** that rely heavily on organic traffic. This can lower search rankings despite having good visual content for users.

3. P75 Effect – Even Fast Users Suffer

What P75 means:
• Core Web Vitals measure at the **75th percentile**, not average.
• This means: **25% of page loads are slow** — for example, LCP ≥ 3.9s.

Now imagine a user visits **5 pages** on your site:

- What's the probability **all 5 pages are fast**?
  → 0.75^5 = 0.237 → **Only 23.7%**
- So, what's the chance they hit **at least one slow page**?
  → 1 - 0.237 = 0.763 → **76.3%**

Implication:
Even if **just 1 in 4 pages is slow**,
Most users (76.3%) will hit a bad page when browsing just 5 pages.

Why it matters:
• Users remember **the slowest page**, not the fastest.
• Slow pages lead to **frustration**, **loss of trust**, and **higher bounce** — even for fast devices on fast networks.
• In multi-page apps like **Milo**, this compounds quickly.

Adobe's Real-World Results:

- **LCP dropped** from 7.2s → 3.4s
- **TTI dropped** from 33.3s → 4.5s
- **Mobile engagement** rose **35%**, bounce rate fell **6%**, and time on page rose **21%**
- A 2022 survey shows: **1 in 2 users abandon sites** that take over 6s to load.

Summary:
• 25% of pages slow → sounds okay in isolation
• But in a 5-page session → 76% chance of poor experience
• So even "fast" users are **not safe** from bad UX

---

4. Inefficient JavaScript Usage

What CSR does:

- Milo triggers multiple cascading network requests per block instead of shipping a single large JS bundle.
- Client must:
  - Download the JS,
  - Parse it,
  - Execute it before showing anything meaningful.

Why it's bad:

- Increases:
  - **Time to Interactive (TTI)** – when page becomes usable.
  - **Total Blocking Time (TBT)** – JS blocks the main thread.
- JS parsing overwhelms CPU, Page becomes janky or unresponsive.

5. Wasted Execution Time:

CSR doesn't just depend on JS execution — but a full **orchestration chain**, including:

- Logic to decide **which components or routes to load**,
- Time to load **external resources** like styles/images,
- Time to **paint** the DOM after layout & hydration.

Breakdown:

• 2–3s: JS file discovery + parsing per block,
• 3–4s: Block-specific data fetches
• 2–3s: DOM updates, font/image loads, and interactivity setup.

Result: 10–15s before the user can meaningfully use the page. That's *huge friction*, especially on first-time visits.

---

6. LCP Is Delayed by Design

In CSR, **LCP (Largest Contentful Paint)** — the biggest visual element — only shows up **after**:

- The JS is fully executed,
- Data is fetched,
- DOM structure evolves as blocks are decorated dynamically,
- Styles are applied,
- Images load and paint.

This makes LCP timing unpredictable and non-deterministic, which:

- Hurts Core Web Vitals scores,
- Makes the page feel "slow" to users,
- Results in inconsistent rendering across users/devices.

---

7. Harder to Optimize the Critical Rendering Path

In CSR:

- You can't predefine the *rendering priority* of content.
- The browser **must wait for JS to execute** before it knows:
    - What elements to paint,
    - What to lazy-load,
    - What to defer.

Why this is problematic:

- You lose control of the **critical rendering path**, unlike SSR/SSG where:
    - HTML is delivered fully formed,
    - CSS and images can be prioritized,

- Render-blocking elements are minimized.

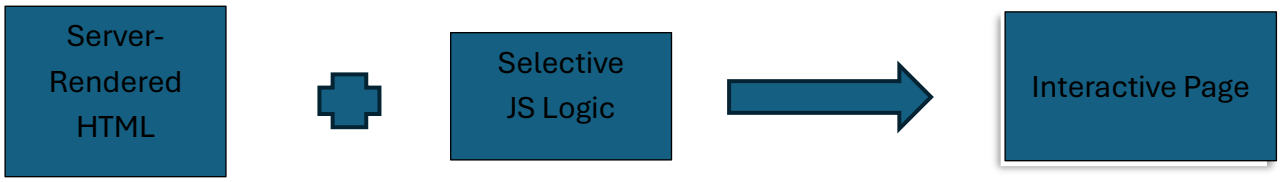
Conclusion: Browser is “blind” until JS hydration finishes — making fine-grained performance tuning very difficult.

	Good	Needs Improvement
FCP	[0, 1800ms]	(1800ms, 3000ms]
LCP	[0, 2500ms]	(2500ms, 4000ms]
CLS	[0, 0.1]	(0.1, 0.25]
INP	[0, 200ms]	(200ms, 500ms]
TTFB (experimental)	[0, , 800ms]	(800ms, 1800ms]

OBJECTIVES

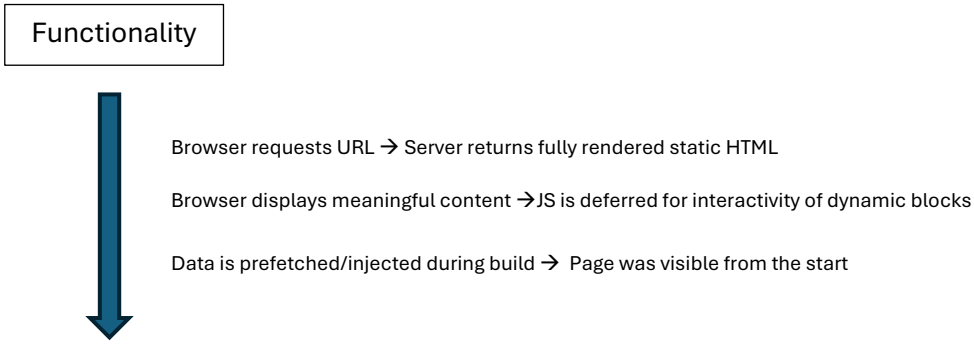
- Improve page load speed and SEO by pre-rendering HTML via SSR.
- Reduce unnecessary JavaScript execution on non-interactive components.
- Enable fine-grained hydration only where interactivity is needed.
- Maintain compatibility with existing Milo blocks and workflows.

PLAN



1. Static Site Generation(SSG) with Artemis

Static Site Generation (SSG) means HTML is rendered once at build time, not during each user request. **Artemis**, Milo’s headless rendering engine, uses Puppeteer to output **fully populated HTML pages ahead of time**, which are served instantly on request.



WHY Static Site Generation?

• **Fixing Slower Initial Loads (FCP/LCP)**
→ SSG delivers fully rendered HTML upfront, reducing time to first paint and making content visible instantly.

• **Improving SEO Discoverability**
Static HTML is crawlable without JavaScript, improving search indexing.

• **Reducing Execution Cost**
→ Rendering and data-fetching happen during build time, so client devices are spared the overhead of heavy JavaScript execution.

• **Stabilizing LCP (Making It Deterministic)**
→ Since content is already in the HTML, large visual elements like hero banners load predictably, improving Core Web Vitals.

• **Enabling Fine-Grained Control Over Rendering Path**
→ Developers can control the order and priority of what loads first, optimizing both perceived performance and interactivity.

2. Modular Hydration

Only hydrate what's interactive. Let static content stay static.

Why Modular Hydration?

• Loading JS for every block is wasteful if not all blocks are interactive.
• Static content shouldn't block page interactivity.

• Reduces JavaScript bundle size and parsing cost
• Accelerates **TTI (Time to Interactive)**
• Makes hydration lightweight and scoped

• Developers annotate interactive elements with @hydrate
• A lightweight runtime parses and hydrates only those blocks
• Leverages **Babel Parser** to extract hydration-annotated logic at build time

3. Babel Parsing & Compilation

Babel is used as a JavaScript parser and compiler during the build process.

• Parses JS files to detect blocks marked with custom decorators (e.g. @hydrate)
• Transforms modern JS into browser-compatible formats
• Helps isolate and extract just the logic needed for hydration
• Ensures compatibility with older browsers while keeping build lean

Why this approach?
We're using a compiler (via Babel) to **retain Milo's architecture** without rewriting blocks for the server — enabling progressive enhancement **without disrupting the existing client-side framework**.

4. Headless Rendering using Puppeteer

- **Why:**
  Simulates browser behavior in a server environment for accurate rendering.
- **What It Solves:**
  - Ensures full layout and visual correctness in the rendered HTML.
  - Enables reuse of rendered output across caching layers.
    Artemis uses **Puppeteer** to render pages in a headless Chromium instance

IMPLEMENTATION

1. ***Detect Block Entry Points***
   Identify Milo blocks under libs/blocks/ that may require hydration logic.

2. ***Extract Hydration Annotations***
   Parse @hydrate and @hydrate.class markers from source files using extract-all.js

3. ***Transform Hydration Code***
   Convert extracted @hydrate annotations into explicit window.__hydrate__ task registrations using transform-hydration.js, ensuring each task has a unique ID and is ready for runtime execution.

4. ***Configure Components***
   Copy original blocks and inject hydration-specific code and runtime hooks into the duplicated files via build.js.

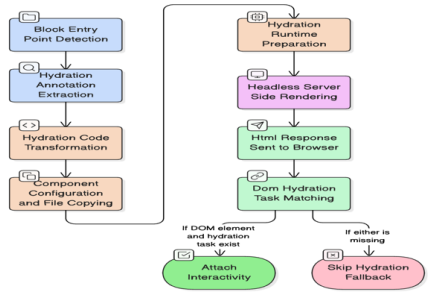
5. ***Prepare Hydration Runtime***
   Initialize hydration-runtime.js, which maintains a task registry (window.__hydrate__) for all extracted hydration handlers.

6. ***Server-Side Render (SSR)***
   Use index.js with Puppeteer to render the page in a headless browser, outputting fully formed HTML and hydration metadata.

7. ***Send HTML Response***
   Serve the rendered HTML to the browser, including hydration placeholders and a script to evaluate window.__hydrate__.

8. ***Match DOM & Hydration Tasks***
   On client load, hydration runtime matches elementId from the DOM with taskId in the hydration map.

9. ***Attach Interactivity or Skip***
   If both task and element match, bind behavior (e.g., event listeners); if not, skip silently, preserving performance.



TECHNOLOGIES USED :

Node.js	Runtime environment for executing server-side scripts including build, extraction, and rendering tasks.
Puppeteer	Headless browser used by Artemis to perform server-side rendering and simulate DOM for HTML snapshotting.
Babel Parser	JavaScript parser used to extract and analyze hydration annotations (@hydrate) from source code.
Artemis	Internal SSR engine at Adobe used to generate pre-rendered HTML from Milo blocks.
JavaScript (ES6+)	Core language used for both block logic and hydration annotations.
HTML/CSS	Structural and styling languages for blocks and decorated content.
Milo Framework	Adobe's in-house content-first framework based on modular block architecture.
Git & GitHub	Version control and collaboration for project tracking and code integration.

SCHEDULE - 10 Week Implementation Plan

Week	Milestones & Activities	Status
Week 1	• Detection of Milo block entry points in libs/blocks/; • Extraction of @hydrate and @hydrate.class annotations using extract-all.js	Done ✅
Week 2	• Transformation of extracted hydration logic via transform-hydration.js; • Setup of component-level configuration and file duplication through build.js	Done ✅
Week 3	• Development of hydration-runtime.js to manage window.__hydrate__; • Integration of Artemis SSR engine and Puppeteer setup	Done ✅
Week 4	• Rendering of static HTML using headless browser (Puppeteer); • DOM element and hydration task ID matching on client-side load;	Done ✅
Week 5	• Hydration of standalone JS blocks (accordion, tabs, editorial card, etc.); • Fallback-safe: Skipping hydration if either the DOM or task is missing	Done ✅
Week 6	• Hydration of standalone JS blocks (modal,editorial card, aside,hero-marquee etc.); • Selective script injection and runtime tracking	Done ✅
Week 7	• Inline script injection for global navigation interactivity • Config declaration for implementing unav functionality •Documentation	Done ✅
Week 8	• Handling class-based blocks (global navigation, footer) hydration; • Merch card hydration	Pending
Week 9	• Performance benchmarking (LCP, CLS, TTI, TBT) using Lighthouse/Web Vitals;	Pending
Week 10	• Final Demo and post-implementation review	Pending

Development Roadblocks

1. Population of `data-feds` for Class Hydration

- **Issue:**
  Class-based hydration requires the presence of a data-feds attribute that maps hydration task IDs.
  Hypothesis:
  This attribute was not generated dynamically and had to be manually added, which is error-prone and not scalable.
- **Resolution:**

2. HTML Attributes Conflicting with Client-Side Interactivity

- **Issue:**
  When pages were pre-rendered using Puppeteer, attributes like data-mouseEvent="true" were already set in the final HTML. This bypassed JavaScript condition checks, preventing interactivity from being re-applied (e.g., in **Editorial Card's** hover-play functionality).
- **Resolution:**
  Removed or reset the interfering attributes from the static HTML output to allow client-side JS to re-evaluate and re-bind the appropriate event listeners.

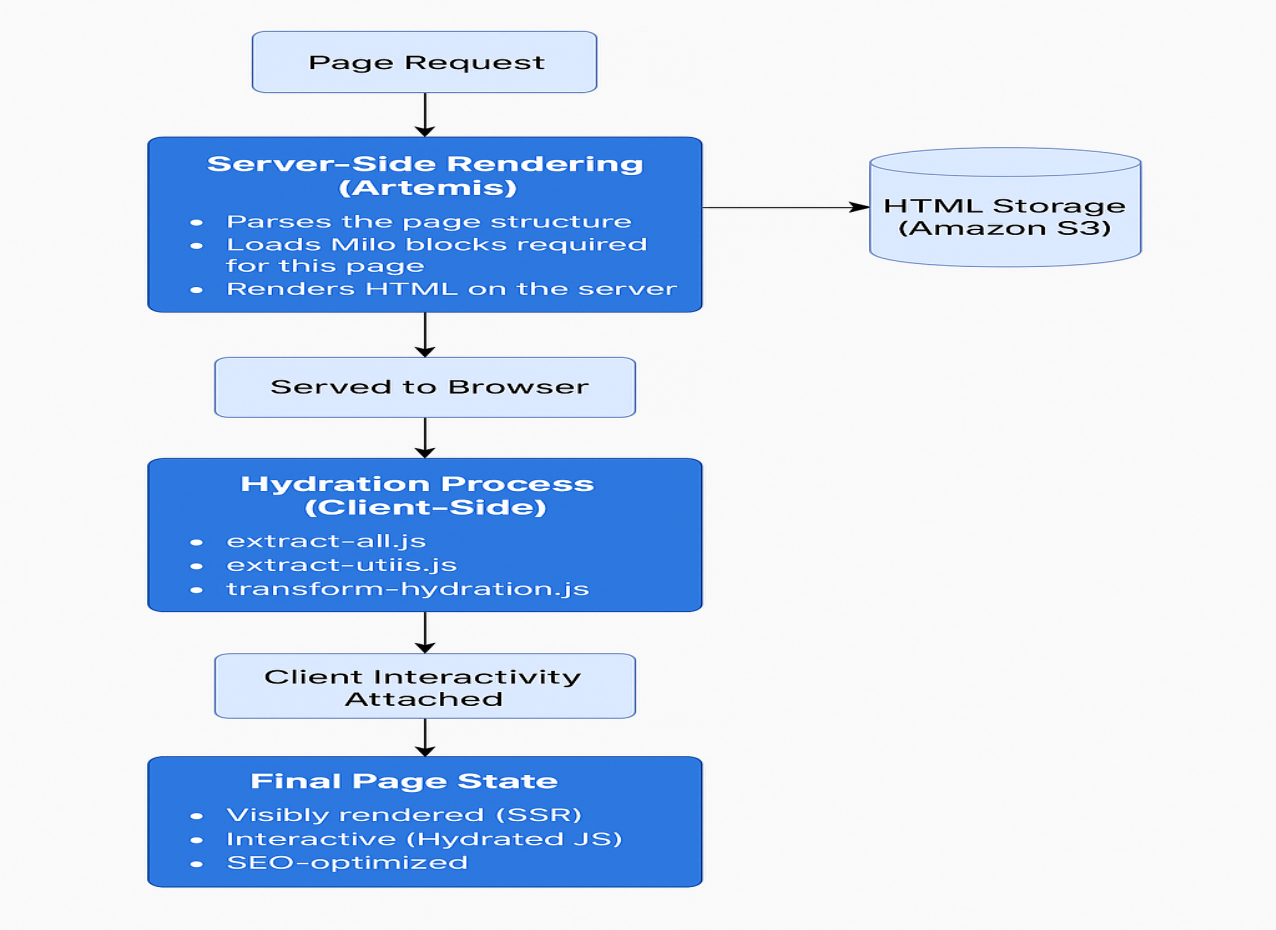
3. Video Interactivity Loss Due to Missing Anchor Decoration

- **Issue:**
  In blocks like **hero-marquee** and **aside**, interactivity depends on <a> tags being transformed into <video> elements via decorateAnchor() and applyAccessibilityEvents(). However, Puppeteer rendered HTML already had <video>, skipping the decoration logic.
- **Resolution:**
  Explicitly invoked applyAccessibilityEvents() post-render to manually bind the required video event handlers and restore expected interactivity behavior.

4. Poor LCP Performance Despite SSR

- **Issue**:
  The marquee block's LCP on the SSR-rendered page was **worse than on the live Photoshop production page** — even though LCP reduction was the primary goal.
  An image with multiple viewport variants took **over 5 seconds** to load in the headless version, compared to **~1.7s** on production.
- **Analysis**:
  - The headless browser loads **all image variants in parallel**, saturating available bandwidth — unlike the client which loads one responsive image.
  - JavaScript and CSS consume CPU but not much bandwidth; large images are the true bottleneck.
  - Design follows a **client-side body-hiding model**, making LCP worse under SSR.
- **Resolution**:

ARTEMIS V2 WORKING :



BEFORE AND AFTER PERFORMANCE COMPARISON :

LEARNINGS: